Computational Learning Theory

Lecture Notes

Topics Covered

1. Sample Complexity
2. Finite and Infinite Hypothesis Spaces
3. Mistake Bound Model

Includes real-time examples and 3 hands-on Python logic workouts per topic

Table of Contents

1. Sample Complexity

1.1 What Is Sample Complexity?

Sample complexity answers a very practical question every machine learning engineer faces: "How much training data do I actually need?" Formally, in the Probably Approximately Correct (PAC) learning framework, the sample complexity is the minimum number of training examples m required so that a learning algorithm outputs a hypothesis h that is approximately correct (error at most ε) with high probability (confidence at least 1 - δ).

The two knobs we control are ε (epsilon), the accuracy we demand, and δ (delta), the confidence we demand. Tightening either knob — wanting a smaller error, or wanting higher certainty — always costs us more training examples.

1.2 The PAC Bound for a Consistent Learner (Finite H)

If the hypothesis space H is finite and the learner always outputs a hypothesis consistent with the training data (zero training error), then the number of examples needed so that, with probability at least 1 - δ, every consistent hypothesis has true error below ε, is:

$$m \geq (1 / \varepsilon) \cdot (\ln|H| + \ln(1/\delta))$$

Notice the dependence is only logarithmic in |H|, the size of the hypothesis space. This is the key insight: even if H has millions of hypotheses, we only need a handful more examples — not millions more — to keep them all in check.

1.3 Real-Time Example: Email Spam Filter

> **Scenario:** A startup is building a spam filter. Their hypothesis space is a set of 1,000 candidate rule-based classifiers (e.g. "flag if subject contains 'FREE' AND sender is unknown"). They want the deployed filter to be wrong on at most 2% of emails (ε = 0.02), and they want this guarantee to hold with 95% confidence (δ = 0.05).

Plugging into the bound:

$$m \geq (1/0.02) \cdot (\ln(1000) + \ln(1/0.05)) \approx 50 \cdot (6.91 + 3.00) \approx 495 \text{ emails}$$

So roughly 500 correctly labelled training emails are enough to guarantee, with 95% confidence, that any rule from their 1,000-rule hypothesis space that fits the training data perfectly will misclassify at most 2% of future emails. This is why data-hungry deep nets and simple rule-based filters have wildly different data appetites — the size (and structure) of H drives everything.

1.4 Logic Workouts — Sample Complexity

Logic Workout 1: PAC Sample Size Calculator

Goal: *Given $|H|$, ε, and δ, compute the minimum number of training examples required.*

```
import math

def pac_sample_complexity(hypothesis_space_size, epsilon, delta):
    """Return the minimum number of examples m for a consistent
    learner over a finite hypothesis space H."""
    m = (1 / epsilon) * (math.log(hypothesis_space_size) + math.log(1 / delta))
    return math.ceil(m)

# --- Try it: spam filter example ---
H_size = 1000       # number of candidate rules
epsilon = 0.02       # want <= 2% true error
delta = 0.05          # want >= 95% confidence

m = pac_sample_complexity(H_size, epsilon, delta)
print(f"Minimum training examples needed: {m}")

# --- Explore the trade-off ---
for eps in [0.1, 0.05, 0.02, 0.01]:
    print(f"epsilon={eps:<5} -> m={pac_sample_complexity(H_size, eps, delta)}")
```

Logic Workout 2: Learning Curve Simulator

Goal: *Simulate how the true error of the worst surviving hypothesis shrinks as training set size grows.*

```
import random

def simulate_learning_curve(true_h, hypothesis_pool, max_examples, trials=200):
    """For increasing numbers of training examples, measure how often
    a 'bad' hypothesis (error > 0.1) survives as consistent."""
    results = []
    for m in range(10, max_examples + 1, 10):
        bad_survivals = 0
        for _ in range(trials):
            data = [random.random() for _ in range(m)]
            # a hypothesis 'survives' if it agrees with true_h on all m points
            for h_error in hypothesis_pool:
                if h_error > 0.1:
                    # probability this bad hypothesis stays consistent by chance
                    if all(random.random() > h_error for _ in range(m)):
                        bad_survivals += 1
                        break
        results.append((m, bad_survivals / trials))
    return results

hypothesis_pool = [0.01, 0.05, 0.15, 0.25, 0.4]   # true error rates of candidate h's
curve = simulate_learning_curve(true_h=0.0, hypothesis_pool=hypothesis_pool,
max_examples=200)

for m, frac_bad in curve[::4]:
    print(f"m={m:<4} fraction of trials where a bad hypothesis survived:
{frac_bad:.3f}")
```

Logic Workout 3: Dataset Sufficiency Checker

Goal: *Given a real dataset size, check whether it satisfies the PAC bound for a target (epsilon, delta, |H|), and report the achievable epsilon otherwise.*

```
import math

def is_sufficient(dataset_size, hypothesis_space_size, epsilon, delta):
    required = (1 / epsilon) * (math.log(hypothesis_space_size) + math.log(1 /
delta))
    return dataset_size >= required, math.ceil(required)

def achievable_epsilon(dataset_size, hypothesis_space_size, delta):
    """Given how much data we actually have, solve the bound for epsilon."""
    return (math.log(hypothesis_space_size) + math.log(1 / delta)) / dataset_size

dataset_size = 300
H_size = 1000
epsilon = 0.02
delta = 0.05

ok, required = is_sufficient(dataset_size, H_size, epsilon, delta)
print(f"Have {dataset_size} examples, need {required} -> sufficient? {ok}")

if not ok:
    eps_reachable = achievable_epsilon(dataset_size, H_size, delta)
    print(f"With only {dataset_size} examples, the best guaranteed epsilon is
{eps_reachable:.4f}")
```

2. Finite and Infinite Hypothesis Spaces

2.1 Finite Hypothesis Spaces

A hypothesis space H is finite when it can be enumerated — a Boolean formula built from a fixed set of literals, a decision list of bounded length, or any classifier chosen from a fixed, countable catalogue. For finite H, |H| appears directly (inside a logarithm) in the PAC sample complexity bound from Section 1, which makes the analysis clean: we can literally count the hypotheses.

Example: for n Boolean features, the space of conjunctions of literals (each feature can appear positively, negatively, or not at all) has size 3^n. Even though this grows exponentially in n, $\ln(3^n) = n \cdot \ln 3$ grows only linearly — so sample complexity stays very manageable.

2.2 Infinite Hypothesis Spaces and VC Dimension

Many practical hypothesis spaces are infinite: linear separators in ^{d}, neural networks with real-valued weights, decision boundaries defined by continuous thresholds. Here $|H| = \infty$ and the finite-H bound is useless. The fix is the Vapnik–Chervonenkis (VC) dimension, VC(H), which measures the expressive "capacity" of H by asking: what is the largest number of points that H can shatter (label in every possible way)?

$$m \geq (1/\varepsilon) \cdot (4 \cdot \log (2/\delta) + 8 \cdot VC(H) \cdot \log (13/\varepsilon))$$

VC(H) plays exactly the role that ln|H| played before: it is a single number capturing how rich the hypothesis class is, and it lets us reason about sample complexity even when there are infinitely many hypotheses. For linear classifiers in ^{d}, VC(H) = d + 1.

2.3 Real-Time Example: Rule-Based vs. Linear Diagnosis Models

Finite case: A clinic uses a symptom checklist with 10 boolean symptoms (fever, cough, fatigue, …) and considers only conjunctions of these symptoms as candidate diagnostic rules. $|H| = 3^{10} \approx 59{,}049$ — finite and enumerable, so the log|H| bound applies directly.

Infinite case: A research lab instead fits a linear model over continuous vitals (temperature, heart rate, blood oxygen). The weight vector lives in ^{3}, so there are infinitely many possible decision boundaries. Sample complexity must be analyzed via VC(H) = 3 + 1 = 4 instead of counting hypotheses.

2.4 Logic Workouts — Finite and Infinite Hypothesis Spaces

Logic Workout 1: Enumerating a Finite Boolean Hypothesis Space

Goal: *Generate all conjunctions of literals over n boolean features and confirm |H| = 3^n.*

```
from itertools import product

def enumerate_conjunctions(n):
    """Each feature can be: absent (0), positive literal (1), negative literal (-
```

```
1)."""
    hypotheses = list(product([0, 1, -1], repeat=n))
    return hypotheses

n = 4
H = enumerate_conjunctions(n)
print(f"Features: {n}")
print(f"|H| computed  = {len(H)}")
print(f"|H| = 3^n     = {3 ** n}")
print("Sample hypothesis:", H[10], " (0=absent, 1=positive literal, -1=negative
literal)")
```

Logic Workout 2: VC-Dimension Based Sample Complexity for Linear Classifiers

Goal: *Compute the sample complexity bound for linear separators in R^d using VC(H) = d + 1, and compare against a finite-H style bound to see why VC dimension is needed.*

```
import math

def vc_sample_complexity(vc_dim, epsilon, delta):
    m = (1 / epsilon) * (4 * math.log2(2 / delta) + 8 * vc_dim * math.log2(13 /
epsilon))
    return math.ceil(m)

for d in [2, 3, 5, 10]:
    vc_dim = d + 1                # linear classifier in R^d
    m = vc_sample_complexity(vc_dim, epsilon=0.05, delta=0.05)
    print(f"R^{d}: VC(H)={vc_dim:<3} -> requires m={m} examples")

print("\nNote: |H| is infinite here (real-valued weights), so ln|H| is undefined —")
print("VC dimension is what makes the bound computable at all.")
```

Logic Workout 3: Shattering Checker

Goal: *Test whether a hypothesis class (intervals on the real line) can shatter a given set of points, to empirically find its VC dimension.*

```
from itertools import product

def interval_classify(point, lo, hi):
    return 1 if lo <= point <= hi else 0

def can_shatter(points, candidate_intervals):
    """Check if every labeling of `points` is achievable by some interval."""
    n = len(points)
    all_labelings = list(product([0, 1], repeat=n))
    achievable = set()
    for lo, hi in candidate_intervals:
        labeling = tuple(interval_classify(pt, lo, hi) for pt in points)
        achievable.add(labeling)
    return all(lab in achievable for lab in all_labelings), len(achievable),
len(all_labelings)

points_2 = [1.0, 2.0]                      # try to shatter 2 points
candidates = [(lo, hi) for lo in range(0, 4) for hi in range(lo, 5)]
shattered, achievable_ct, total_ct = can_shatter(points_2, candidates)
print(f"2 points: shattered={shattered} ({achievable_ct}/{total_ct} labelings
```

```
achievable)")

points_3 = [1.0, 2.0, 3.0]                  # try to shatter 3 points
shattered, achievable_ct, total_ct = can_shatter(points_3, candidates)
print(f"3 points: shattered={shattered} ({achievable_ct}/{total_ct} labelings
achievable)")
print("\n=> Intervals shatter 2 points but not 3, confirming VC(intervals) = 2.")
```

3. Mistake Bound Model

3.1 What Is the Mistake Bound Model?

The mistake bound model is an online learning framework, in contrast to the batch, offline setting used by PAC learning. The learner sees examples one at a time, predicts a label, then is told the true label. Every time the prediction is wrong, that counts as a mistake. We care about an upper bound on the total number of mistakes the learner can ever make, no matter how the examples are ordered or chosen (even adversarially).

This model is a natural fit for streaming or continuously-arriving data — fraud detection, ad click prediction, live sensor monitoring — where the model must act immediately and cannot wait to collect a batch of labelled data first.

3.2 The Halving Algorithm

The Halving Algorithm maintains a "version space" of all hypotheses in H still consistent with every example seen so far. To predict, it takes a majority vote among the surviving hypotheses. Whenever the algorithm makes a mistake, at least half of the surviving hypotheses must have voted wrong — and are removed. This gives a strikingly small worst-case mistake bound:

$$Mistakes(Halving) \leq \log \quad |H|$$

Just like the PAC bound, this depends only logarithmically on |H|. A hypothesis space with a million candidates yields a worst-case mistake bound of only about 20.

3.3 The Perceptron Mistake Bound

For linearly separable data with margin γ (the smallest distance from any point to the true separating hyperplane, after normalizing feature vectors to have norm ≤ R), the Perceptron algorithm's worst-case mistake bound is:

$$Mistakes(Perceptron) \leq (R / \gamma)^2$$

This bound does not depend on the number of features or the number of training examples at all — only on the geometry of the data (how separable it is). Easier problems (large margin γ) provably need far fewer correction steps.

3.4 Real-Time Example: Live Credit-Card Fraud Detection

Scenario: A payment processor scores each transaction the instant it happens (online setting, not batch). The system starts with a hypothesis space of 200,000 simple fraud rules (combinations of thresholds on amount, location-mismatch, and transaction velocity). Using the Halving Algorithm reasoning, the worst-case number of misclassified transactions before the system converges on the truth is bounded by $\log_2(200\{,\}000) \approx 17.6$, i.e. at most 18 mistakes — regardless of how many millions of transactions ultimately arrive. In practice a Perceptron-

style linear model is used instead, and its mistake bound $(R/\gamma)^2$ tells the team exactly how many corrective updates to expect if genuine and fraudulent transactions are well separated in feature space.

3.5 Logic Workouts — Mistake Bound Model

Logic Workout 1: Halving Algorithm Simulator

Goal: *Implement the Halving Algorithm over a small boolean hypothesis space and empirically verify the mistake count never exceeds log2|H|.*

```
import math, random
from itertools import product

def halving_algorithm(hypotheses, examples):
    version_space = list(hypotheses)   # all still-consistent hypotheses
    mistakes = 0
    for x, true_label in examples:
        votes = [h(x) for h in version_space]
        prediction = 1 if votes.count(1) >= votes.count(0) else 0
        if prediction != true_label:
            mistakes += 1
        version_space = [h for h in version_space if h(x) == true_label]
    return mistakes

# Hypothesis space: all boolean 'AND of literals' functions over 3 features
def make_conjunction(signs):
    def h(x):
        for xi, s in zip(x, signs):
            if s == 1 and xi != 1: return 0
            if s == -1 and xi != 0: return 0
        return 1
    return h

hypotheses = [make_conjunction(signs) for signs in product([0, 1, -1], repeat=3)]
true_h = make_conjunction((1, -1, 0))     # feature0 AND NOT feature1

examples = [(x, true_h(x)) for x in product([0, 1], repeat=3)] * 5
random.shuffle(examples)

mistakes = halving_algorithm(hypotheses, examples)
bound = math.log2(len(hypotheses))
print(f"|H| = {len(hypotheses)}, log2|H| = {bound:.2f}")
print(f"Mistakes made: {mistakes}  (must be <= {math.floor(bound)})")
```

Logic Workout 2: Perceptron With Mistake Counting

Goal: *Implement the online Perceptron algorithm, count mistakes on linearly separable data, and compare against the theoretical (R/gamma)^2 bound.*

```
import numpy as np

def perceptron_online(X, y, max_epochs=50):
    w = np.zeros(X.shape[1])
    mistakes = 0
    for epoch in range(max_epochs):
        epoch_mistakes = 0
        for xi, yi in zip(X, y):
```

```
            pred = 1 if np.dot(w, xi) >= 0 else -1
            if pred != yi:
                w += yi * xi
                mistakes += 1
                epoch_mistakes += 1
        if epoch_mistakes == 0:
            break
    return w, mistakes

# Linearly separable toy dataset
np.random.seed(0)
X_pos = np.random.randn(20, 2) + np.array([3, 3])
X_neg = np.random.randn(20, 2) + np.array([-3, -3])
X = np.vstack([X_pos, X_neg])
y = np.array([1] * 20 + [-1] * 20)

w, mistakes = perceptron_online(X, y)

R = np.max(np.linalg.norm(X, axis=1))
margin = min(y[i] * np.dot(w, X[i]) / np.linalg.norm(w) for i in range(len(X)))
bound = (R / margin) ** 2 if margin > 0 else float('inf')

print(f"Total mistakes made: {mistakes}")
print(f"R (max norm) = {R:.2f}, estimated margin = {margin:.2f}")
print(f"Theoretical mistake bound (R/gamma)^2 ≈ {bound:.2f}")
```

Logic Workout 3: Weighted Majority Algorithm

Goal: *Track a pool of experts, downweight wrong ones, and compare the algorithm's total mistakes to the best single expert's mistakes.*

```
import random

def weighted_majority(expert_predictions, true_labels, beta=0.5):
    n_experts = len(expert_predictions[0])
    weights = [1.0] * n_experts
    algo_mistakes = 0
    expert_mistakes = [0] * n_experts

    for t, true_label in enumerate(true_labels):
        preds = expert_predictions[t]
        vote_for_1 = sum(w for w, p in zip(weights, preds) if p == 1)
        vote_for_0 = sum(w for w, p in zip(weights, preds) if p == 0)
        prediction = 1 if vote_for_1 >= vote_for_0 else 0

        if prediction != true_label:
            algo_mistakes += 1
        for i, p in enumerate(preds):
            if p != true_label:
                weights[i] *= beta          # penalize wrong experts
                expert_mistakes[i] += 1
    return algo_mistakes, expert_mistakes

random.seed(1)
T = 200
true_labels = [random.randint(0, 1) for _ in range(T)]

# 5 experts with different accuracy levels
```

```
accuracies = [0.55, 0.65, 0.75, 0.90, 0.50]
expert_predictions = []
for label in true_labels:
    row = [label if random.random() < acc else 1 - label for acc in accuracies]
    expert_predictions.append(row)

algo_mistakes, expert_mistakes = weighted_majority(expert_predictions, true_labels)
best_expert_mistakes = min(expert_mistakes)

print(f"Weighted Majority total mistakes: {algo_mistakes}")
print(f"Per-expert mistakes: {expert_mistakes}")
print(f"Best single expert mistakes: {best_expert_mistakes}")
print("=> Weighted Majority stays competitive with the best expert in hindsight.")
```

End of Lecture Notes